COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 212 - Automating Server Management | **SECTION:** | CPE31S2 |
| **DATE PERFORMED:** | 10/16/2026 | **DATE SUBMITTED:** | 10/18/2026 |
| **NAME:** | Charles Vence Espinas | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 12** **Build a Sample Web App in a Docker Container**

**1. PROCEDURE OUTPUT**

# Lab - Build a Sample Web App in a Docker Container

## Objectives

**Part 1: Launch the DEVASC VM**

**Part 2: Create a Simple Bash Script**

**Part 3: Create a Sample Web App**

**Part 4: Configure the Web App to Use Website Files**

**Part 5: Create a Bash Script to Build and Run a Docker Container**

**Part 6: Build, Run, and Verify the Docker Container**

## Background / Scenario

In this lab, you will review basic bash scripting techniques because bash scripting is a prerequisite for the rest of the lab. You will then build and modify a Python script for a simple web application. Next, you will create a bash script to automate the process for creating a Dockerfile, building the Docker container, and running the Docker container. Finally, you will use **docker** commands to investigate the intricacies of the Docker container instance.

## Required Resources

- 1 PC with operating system of your choice
- Virtual Box or VMWare
- DEVASC Virtual Machine

## Instructions

## Part 1: Launch the DEVASC VM

If you have not already completed the **Lab - Install the Virtual Machine Lab Environment**, do so now. If you have already completed that lab, launch the DEVASC VM now.

## Part 2: Create a Simple Bash Script

Bash knowledge is crucial for working with continuous integration, continuous deployment, containers, and with your development environment. Bash scripts help programmers automate a variety of tasks in one script file. In this part, you will briefly review how to create a bash script. Later in the lab, you will use a bash script to automate the creation of a web app inside of a Docker container.

### Step 1: Create an empty bash script file.

Change your working directory to **~/labs/devnet-src/sample-app** and add a new file called **user-input.sh**.

```
devasc@labvm:~$ cd labs/devnet-src/sample-app/
devasc@labvm:~/labs/devnet-src/sample-app$ touch user-input.sh
```

```
[analyst@secOps ~]$ cd labs/devnet-src/sample-app
[analyst@secOps sample-app]$ touch user-input.sh
[analyst@secOps sample-app]$
```

### Step 2: Open the file in the nano text editor.

Use the **nano** command to open the nano text editor.

```
devasc@labvm:~/labs/devnet-src/sample-app$ nano user-input.sh
```

```
[analyst@secOps sample-app]$ nano user-input.sh
[analyst@secOps sample-app]$
```

### Step 3: Add the 'she-bang' to the top of the script.

From here you can enter commands for your bash script. Use the arrow keys to navigate in **nano**. Notice the commands at the bottom (not shown here) for managing the file. The carat symbol (^) indicates that you use the CTRL or Command key on your keyboard. For example, to exit **nano**, type CTRL+X.

Add the 'she-bang' which tells the system that this file includes commands that need to be run in the bash shell.

```
#!/bin/bash
```

**Note**: You can use a graphical text editor or open the file with VS Code. However, you should be familiar with command-line text editors like **nano** and **vim**. Search the internet for tutorials to refresh your skill or learn more about them.

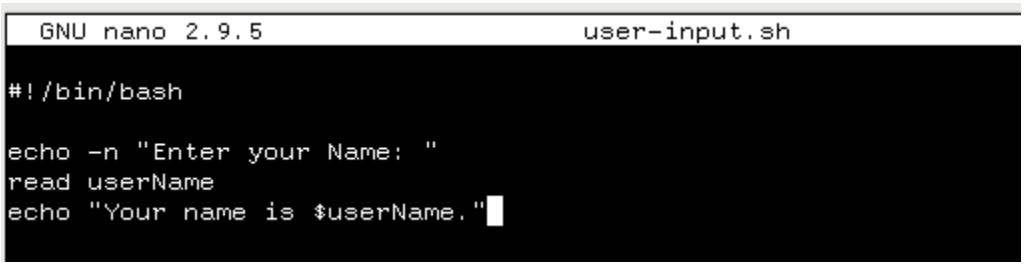
```
  GNU nano 2.9.5                    user-input.sh                    Modifie

#!/bin/bash
```

## Step 4: Add simple bash commands to the script.

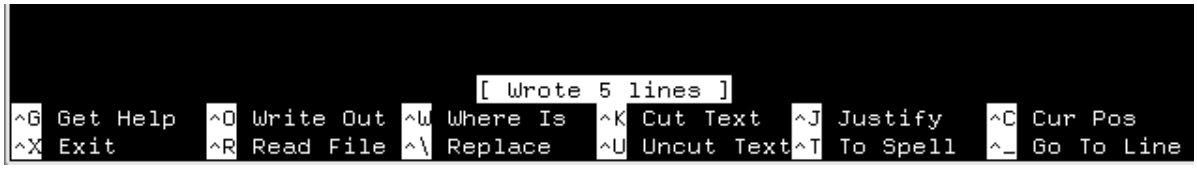
Enter some simple bash commands for your script. The following commands will ask the user for a name, set the name to a variable called **userName**, and display a string of text with the user's name.

```
echo -n "Enter Your Name: "
read userName
echo "Your name is $userName."
```




## Step 5: Exit nano and save your script.

Press **CTRL+X**, then **Y**, then **ENTER** to exit **nano** and save your script.




## Step 6: Run your script from the command line.

You can run it directly from the command line using the following command.

```
devasc@labvm:~/labs/devnet-src/sample-app$ bash user-input.sh
Enter Your Name: Bob
Your name is Bob.
devasc@labvm:~/labs/devnet-src/sample-app$
```




## Step 7: Change the mode of the script to an executable file for all users.

Change the mode of the script to an executable using the **chmod** command. Set the options to **a+x** to make the script executable (x) by all users (a). After using **chmod**, notice permissions have been modified for users, groups, and others to include the "x" (executable).

```
devasc@labvm:~/labs/devnet-src/sample-app$ ls -l user-input.sh
-rw-rw-r-- 1 devasc devasc 84 Jun  7 16:43 user-input.sh

devasc@labvm:~/labs/devnet-src/sample-app$ chmod a+x user-input.sh
```

```
devasc@labvm:~/labs/devnet-src/sample-app$ ls -l user-input.sh
-rwxrwxr-x 1 devasc devasc 84 Jun  7 16:43 user-input.sh
```

```
[analyst@secOps sample-app]$ ls -l user-input.sh
-rw-r--r-- 1 analyst analyst 86 Oct 15 10:34 user-input.sh
[analyst@secOps sample-app]$ chmod a+x user-input.sh
[analyst@secOps sample-app]$ chmod a+x user-input.sh
[analyst@secOps sample-app]$ ls -l user-input.sh
-rwxr-xr-x 1 analyst analyst 86 Oct 15 10:34 user-input.sh
```

### Step 8: Rename the file to remove the .sh extension.

You can rename the file to remove the extension so that users do not have to add .sh to the command to execute the script.

```
devasc@labvm:~/labs/devnet-src/sample-app$ mv user-input.sh user-input
```

```
[analyst@secOps sample-app]$ mv user-input.sh user-input
[analyst@secOps sample-app]$
```

### Step 9: Execute the script from the command line.

Now the script can be run from the command line without the **source** command or an extension. To run a bash script without the source command, you must preface the script with "./".

```
devasc@labvm:~/labs/devnet-src/sample-app$ ./user-input
Enter Your Name: Bob
Your name is Bob.
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ ./user-input
Enter your Name: Charles Vence
Your name is Charles Vence.
[analyst@secOps sample-app]$
```

### Step 10: Investigate other bash scripts.

If you have little or no experience creating bash scripts, take some time to search the internet for bash tutorials, bash examples, and bash games.

## Part 3: Create a Sample Web App

Before we can launch an application in a Docker container, we first need to have the app. In this part, you will create a very simple Python script that will display the IP address of the client when the client visits the web page.

### Step 1: Install Flask and open a port on the DEVASC VM firewall.

Web application developers using Python typically leverage a framework. A framework is a code library to make it easier for developers to create reliable, scalable and maintainable web applications. Flask is a web application framework written in Python. Other frameworks include Tornado and Pyramid.

You will use this framework to create the sample web app. Flask receives requests and then provides a response to the user in the web app. This is useful for dynamic web applications because it allows user interaction and dynamic content. What makes your sample web app dynamic is that it will be displaying the IP address of the client.

**Note**: Understanding Flask functions, methods, and libraries are beyond the scope of this course. It is used in this lab to show how quickly you can get a web application up and running. If you want to learn more, search the internet for more information and tutorials on the Flask framework.

Open a terminal window and import **flask**.

```
devasc@labvm:~/labs/devnet-src/sample-app$ pip3 install flask
```

```
[analyst@secOps sample-app]$ python3 -m pip install flask --user
Collecting flask
  Downloading https://files.pythonhosted.org/packages/cd/77/59df23681f4fd19b7cbbb5e92484d46ad587554f5d490f33ef907e456132/Flask-2.0.3-py3-none-any.whl (95kB)
    100% |████████████████████████████████| 102kB 11.2MB/s
Collecting Jinja2>=3.0 (from flask)
  Downloading https://files.pythonhosted.org/packages/20/9a/e5d9ec41927401e41aea8af6d16e78b5e612bca4699d417f646a9610a076/Jinja2-3.0.3-py3-none-any.whl (133kB)
    100% |████████████████████████████████| 143kB 12.0MB/s
Collecting Werkzeug>=2.0 (from flask)
  Downloading https://files.pythonhosted.org/packages/f4/f3/22afbdb20cc4654b10c98043414a14057cd27fdba9d4ae61cea596000ba2/Werkzeug-2.0.3-py3-none-any.whl (289kB)
    100% |████████████████████████████████| 296kB 6.2MB/s
Collecting click>=7.1.2 (from flask)
  Downloading https://files.pythonhosted.org/packages/4a/a8/0b2ced25639fb20cc1c9784de90a8c25f9504a7f18cd8b5397bd61696d7d/click-8.0.4-py3-none-any.whl (97kB)
    100% |████████████████████████████████| 102kB 14.3MB/s
Collecting itsdangerous>=2.0 (from flask)
  Downloading https://files.pythonhosted.org/packages/9c/96/26f935afba9cd6140216da5add223a0c465b99d0f112b68a4ca426441019/itsdangerous-2.0.1-py3-none-any.whl
Collecting MarkupSafe>=2.0 (from Jinja2>=3.0->flask)
  Downloading https://files.pythonhosted.org/packages/fc/d6/57f9a97e56447a1e340f8574836d3b636e2c14de304943836bd645fa9c7e/MarkupSafe-2.0.1-cp36-cp36m-manylinux1_x86_64.whl
Collecting dataclasses; python_version < "3.7" (from Werkzeug>=2.0->flask)
  Downloading https://files.pythonhosted.org/packages/fe/ca/75fac5856ab5cfa51bbbcefa250182e50441074fdc3f803f6e76451fab43/dataclasses-0.8-py3-none-any.whl
Collecting importlib-metadata; python_version < "3.8" (from click>=7.1.2->flask)
  Downloading https://files.pythonhosted.org/packages/a0/a1/b153a0a4caf7a7e3f15c2cd56c7702e2cf3d89b1b359d1f1c5e59d68f4ce/importlib_metadata-4.8.3-py3-none-any.whl
Collecting typing-extensions>=3.6.4; python_version < "3.8" (from importlib-metadata; python_version < "3.8"->click>=7.1.2->flask)
  Downloading https://files.pythonhosted.org/packages/45/6b/44f7f8f1e110027cf88956b59f2fad776cca7e1704396d043f89effd3a0e/typing_extensions-4.1.1-py3-none-any.whl
Collecting zipp>=0.5 (from importlib-metadata; python_version < "3.8"->click>=7.1.2->flask)
  Downloading https://files.pythonhosted.org/packages/bd/df/d4a4974a3e3957fd1c1fa3082366d7fff6e428ddb55f074bf64876f8e8ad/zipp-3.6.0-py3-none-any.whl
Installing collected packages: MarkupSafe, Jinja2, dataclasses, Werkzeug, typing-extensions, zipp, importlib-metadata, click, itsdangerous, flask
Successfully installed Jinja2-3.0.3 MarkupSafe-2.0.1 Werkzeug-2.0.3 click-8.0.4 dataclasses-0.8 flask-2.0.3 importlib-metadata-4.8.3 itsdangerous-2.0.1 typing-extensions-4.1.1 zipp-3.6.0
You are using pip version 9.0.3, however version 26.2.1 is available.
You should consider upgrading via the 'pip install --upgrade pip' command.
```

## Step 2: **Open the sample_app.py file.**

Open the **sample_app.py** file located in the **/sample-app** directory. You can do this inside VS Code or you can use a command-line text editor like **nano** or **vim**.

```
[analyst@secOps sample-app]$ nano sample_app.py
```

## Step 3: **Add the commands to import methods from flask.**

Add the following commands to import the required methods from the flask library.

```
from flask import Flask
from flask import request
```

```
 GNU nano 2.9.5                          sample_app.py

from flask import Flask
from flask import request
```

## Step 4: **Create an instance of the Flask class.**

Create an instance of the Flask class and name it **sample**. Be sure to use two underscores before and after the "name".

```
sample = Flask(__name__)
```

```
sample = Flask(__name__)
```

## Step 5: **Define a method to display the client IP address.**

Next, configure Flask so that when a user visits the default page (root directory), it displays a message with the IP address of the client.

```
@sample.route("/")
def main():
    return "You are calling me from " + request.remote_addr + "\n"
```

```
@sample.route("/")
def main():
        return "You are calling me from " + request.remote_addr + "\n"
```

Notice the @sample.route("/") Flask statement. Frameworks such as Flask use a routing technique (.route) to refer to an application URL (this not to be confused with network routing). Here the "/" (root directory) is bound to the main() function. So, when the user goes to http://localhost:8080/ (root directory) URL, the output of the return statement will be displayed in the browser.

## Step 6: **Configure the app to run locally.**

Finally, configure Flask to run the app locally at http://0.0.0.0:8080, which is also http://localhost:8080. Be sure to use two underscores before and after "name", and before and after "main".

```
if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
```

```
if __name__ == "__main__":
        sample.run(host="0.0.0.0", port=8080)
```

## Step 7: **Save and run your sample web app.**

Save your script and run it from the command line. You should see the following output which indicates that your “sample-app” server is running. If you do not see the following output or if you receive an error message, check your sample_app.py script carefully.

```
devasc@labvm:~/labs/devnet-src/sample-app$ python3 sample_app.py
 * Serving Flask app "sample-app" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production
deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

```
  GNU nano 2.9.5                        sample_app.py

from flask import Flask
from flask import request

sample = Flask(__name__)

@sample.route("/")
def main():
        return "You are calling me from " + request.remote_addr + "\n"

if __name__ == "__main__":
        sample.run(host="0.0.0.0", port=8080)
```

```
[analyst@secOps sample-app]$ python3 sample_app.py
 * Serving Flask app 'sample_app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://192.168.100.115:8080/ (Press CTRL+C to quit)
```

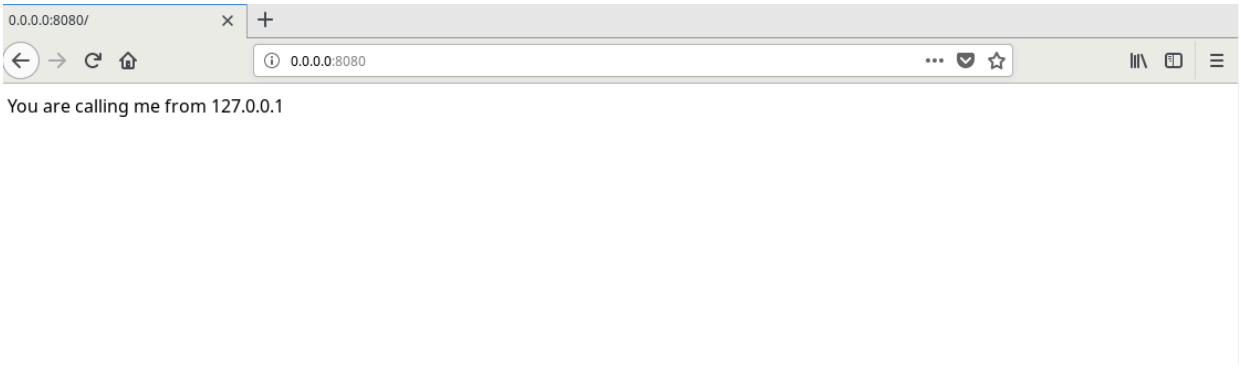
## Step 8: **Verify the server is running.**

You can verify the server is running in one of two ways.

a. Open the Chromium web browser and enter 0.0.0.0:8080 in the URL field. You should get the following output:

**You are calling me from 127.0.0.1**

If you receive an "HTTP 400 Bad Request" response, check your sample_app.py script carefully.




b. Open another terminal window and use the command-line URL tool (cURL) to verify the server's response.

```
devasc@labvm:~/labs/devnet-src/sample-app$ curl http://0.0.0.0:8080
You are calling me from 127.0.0.1
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ curl http://0.0.0.0:8080
You are calling me from 127.0.0.1
[analyst@secOps sample-app]$
```

## Step 9: Stop the server.

Return to the terminal window where the server is running and press CTRL+C to stop the server.

```
[analyst@secOps sample-app]$ python3 sample_app.py
 * Serving Flask app 'sample_app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://192.168.100.115:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [15/Oct/2026 11:13:03] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Oct/2026 11:13:03] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [15/Oct/2026 11:13:03] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [15/Oct/2026 11:14:43] "GET / HTTP/1.1" 200 -
[analyst@secOps sample-app]$
```

# Part 4: Configure the Web App to use Website Files

In this part, build out the sample web app to include an **index.html** page and **style.css** specification. The **index.html** is typically the first page loaded in a client's web browser when visiting your website. The **style.css** is a style sheet used to customize the look of the web page.

## Step 1: Explore the directories that will be used by the web app.

The directories **templates** and **static** are already in the **sample-app** directory. Open the **index.html** and **style.css** to view their contents. If you are familiar with HTML and CSS, feel free to customize these directories and files as much as you like. However, be sure you keep the embedded

**{{request.remote_addr}}** Python code in the **index.html** file as this is the dynamic aspect of the sample web app.

```
devasc@labvm:~/labs/devnet-src/sample-app$ cat templates/index.html
<html>
<head>
    <title>Sample app</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>You are calling me from {{request.remote_addr}}</h1>
</body>
</html>
devasc@labvm:~/labs/devnet-src/sample-app$ cat static/style.css
body {background: lightsteelblue;}
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ cat templates/index.html
<html>
<head>
  <title>Sample app</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <h1>You are calling me from {{request.remote_addr}}</h1>
</body>
</html>

[analyst@secOps sample-app]$ cat static/style.css
body {background: lightsteelblue;}
```

## Step 2: Update the Python code for the sample web app.

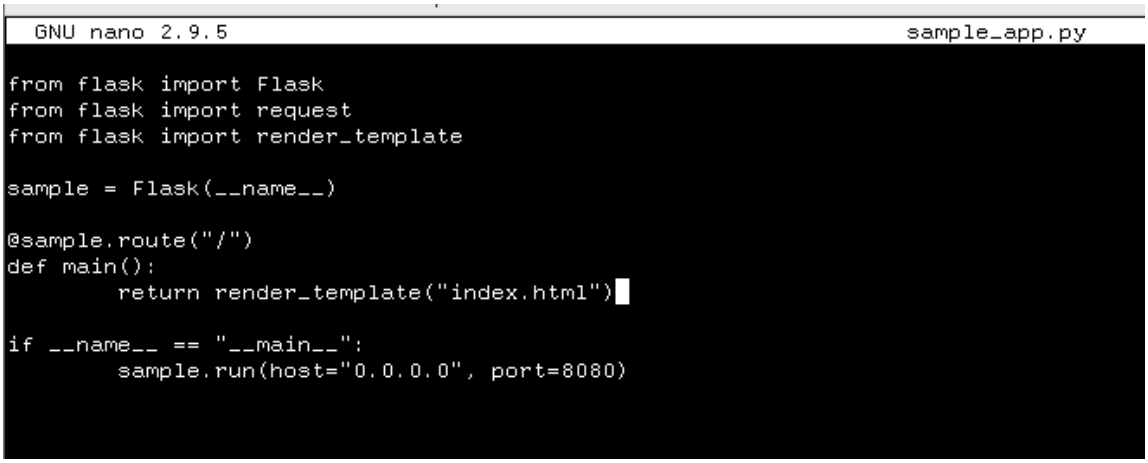
Now that you have explored the basic website files, you need to update the **sample_app.py** file so that it renders the **index.html** file instead of just returning data. Generating HTML content using Python code can be cumbersome, especially when using conditional statements or repeating structures. The HTML file can be rendered in Flask automatically using the render_template function. This requires importing the **render_template** method from the flask library and editing to the **return** function. Make the highlighted edits to your script.

```
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

@sample.route("/")
def main():
    return render_template("index.html")
```

```
if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
```

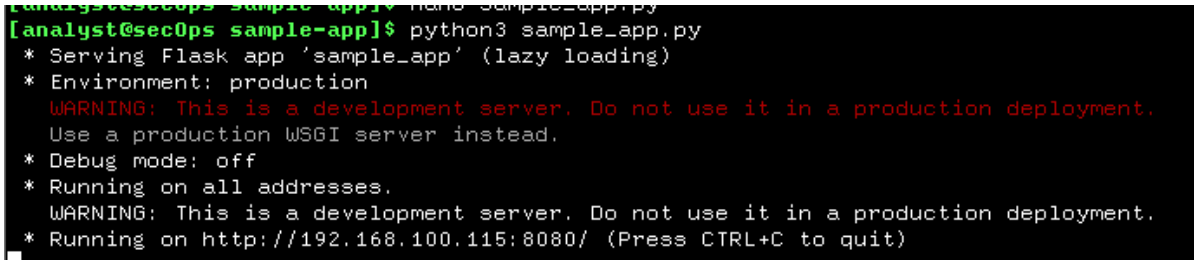



## Step 3: **Save and run your script.**

Save and run your **sampe-app.py** script. You should get output like the following:

```
devasc@labvm:~/labs/devnet-src/sample-app$ python3 sample_app.py
 * Serving Flask app "sample-app" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production
deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

**Note**: If you got Traceback output and an error with the message with something like **OSError: [Errno 98] Address already in use**, then you did not shutdown your previous server. Return to the terminal window where that server is running and press CTRL+C to end the server process. Re-run your script.

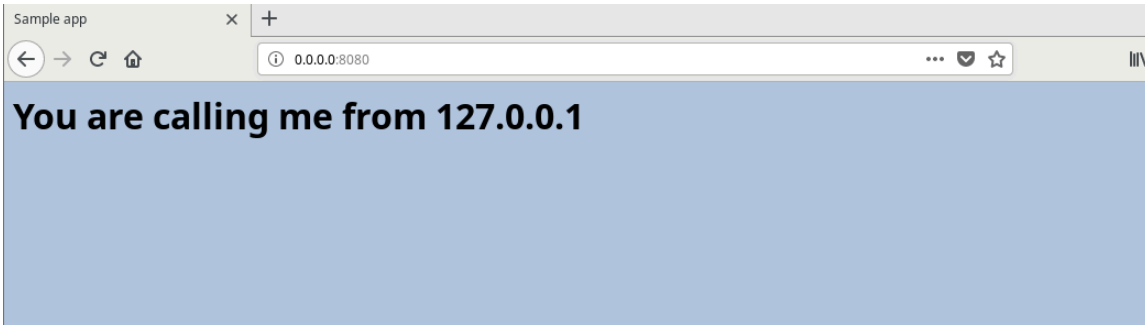



## Step 4: **Verify your program is running.**

Again, you can verify your program is running in one of two ways.

a. Open the Chromium web browser and enter 0.0.0.0:8080 in the URL field. You should get the same output as before. However, your background will be light steel blue and the text will be formatted as H1.

**You are calling me from 127.0.0.1**




b. Open another terminal window and use the **curl** command to verify the server's response. This is where you will see the result of the HTML code rendered automatically using the render_template function. In this case, you will get all the HTML content. However, the dynamic Python code will be replaced with the value for **{{request.remote_addr}}**. Also, notice your prompt will be on the same line as the last line of HMTL output. Press ENTER to get a new line.

```
devasc@labvm:~/labs/devnet-src/sample-app$ curl http://0.0.0.0:8080
<html>
<head>
    <title>Sample app</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>You are calling me from 127.0.0.1</h1>
</body>
</html>devasc@labvm:~/labs/devnet-src/sample-app$
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ curl http://0.0.0.0:8080
<html>
<head>
  <title>Sample app</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <h1>You are calling me from 127.0.0.1</h1>
</body>
</html>
```

## Step 5: **Stop the server.**

Return to the terminal window where the server is running and press CTRL+C to stop the server.

```
[analyst@secOps sample-app]$ python3 sample_app.py
 * Serving Flask app 'sample_app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://192.168.100.115:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [15/Oct/2026 11:29:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Oct/2026 11:29:07] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [15/Oct/2026 11:29:50] "GET / HTTP/1.1" 200 -
```

## Part 5: Create a Bash Script to Build and Run a Docker Container

An application can be deployed on a bare metal server (physical server dedicated to a single-tenant environment) or in a virtual machine, like you just did in the previous Part. It can also be deployed in a containerized solution like Docker. In this part, you will create a bash script and add commands to it that complete the following tasks to build and run a Docker container:

- Create temporary directories to store the website files.
- Copy the website directories and sample_app.py to the temporary directory.
- Build a Dockerfile.
- Build the Docker container.
- Start the container and verify it is running.

### Step 1: Create temporary directories to store the website files.

Open the **sample-app.sh** bash script file in the **~/labs/devnet-src/sample-app** directory. Add the 'she-bang' and the commands to create a directory structure with **tempdir** as the parent folder.

```
#!/bin/bash

mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static
```

```
#!/bin/bash

mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static
```

### Step 2: Copy the website directories and sample_app.py to the temporary directory.

in the **sample-app.sh** file, add the commands to copy the website directory and script to **tempdir**.

```
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.
```

```
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.
```

## Step 3: **Create a Dockerfile.**

In this step, you enter the necessary bash **echo** commands to the **sample-app.sh** file to create a Dockerfile in the **tempdir**. This Dockerfile will be used to build the container.

a. You need Python running in the container, so add the Docker **FROM** command to install Python in the container.

```
echo "FROM python" >> tempdir/Dockerfile
```

```
echo "FROM python" >> tempdir/Dockerfile
```

b. Your **sample_app.py** script needs Flask, so add the Docker **RUN** command to install Flask in the container.

```
echo "RUN pip install flask" >> tempdir/Dockerfile
```

```
echo "RUN pip install flask" >> tempdir/Dockerfile
```

c. Your container will need the website folders and the **sample_app.py** script to run the app, so add the Docker **COPY** commands to add them to a directory in the Docker container. In this example, you will create **/home/myapp** as the parent directory inside the Docker container. Besides copying the sample_app.py file to the Dockerfile, you will also be copying the index.html file from the templates directory and the style.css file from the static directory.

```
echo "COPY  ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY  ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY  sample_app.py /home/myapp/" >> tempdir/Dockerfile
```

```
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
```

d. Use the Docker **EXPOSE** command to expose port 8080 for use by the webserver.

```
echo "EXPOSE 8080" >> tempdir/Dockerfile
```

```
echo "EXPOSE 8080" >> tempdir/Dockerfile
```

e. Finally, add the Docker **CMD** command to execute the Python script.

```
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile
```

```
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile
```

## Step 4: **Build the Docker container.**

Add the commands to the **sample-app.sh** file to switch to the **tempdir** directory and build the Docker container. The **docker build** command **-t** option allows you to specify the name of the container and the trailing period (**.**) indicates that you want the container built in the current directory.

```
cd tempdir
docker build -t sampleapp .
```

```
cd tempdir
docker build -t sampleapp .
```

## Step 5: **Start the container and verify it is running.**

a. Add the **docker run** command to the **sample-app.sh** file to start the container.

```
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
```

The **docker run** options indicate the following:

- **-t** specifies that you want a terminal created for the container so the you can access it at the command line.
- **-d** indicates that you want the container to run in the background and print the container ID when executing the **docker ps -a** command**.**
- **-p** specifies that you want to publish the container's internal port to the host. The first "8080" references the port for the app running in the docker container (our sampleapp). the second "8080" tells docker to use this port on the host. These values do not have to be the same. For example, an internal port 80 to external 800 (**80:800**).
- **--name** specifies first what you want to call the instance of the container (**samplerunning**) and then the container image that the instance will be based on (**sampleapp**). The instance name can be anything you want. However, the image name needs to match the container name you specified in the docker build command (**sampleapp**).

```
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
```

b. Add the **docker ps -a** command to display all currently running Docker containers. This command will be the last one executed by the bash script.

```
docker ps -a
```

```
docker ps -a
```

### Step 6: **Save your bash script.**

```
 GNU nano 2.9.5                                                  sample-app.sh

#!/bin/bash

mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static

cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.

echo "FROM python" >> tempdir/Dockerfile

echo "RUN pip install flask" >> tempdir/Dockerfile

echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile

echo "EXPOSE 8080" >> tempdir/Dockerfile

echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile

cd tempdir
docker build -t sampleapp .

docker run -t -d -p 8080:8080 --name samplerunning sampleapp

docker ps -a
```

## Part 6: Build, Run, and Verify the Docker Container

In this part, you will execute bash script which will make the directories, copy over the files, create a Dockerfile, build the Docker container, run an instance of the Docker container, and display output from the **docker ps -a** command showing details of the container currently running. Then you will investigate the Docker container, stop the container from running, and remove the container.

**Note**: Be sure you stopped any other web server processes you may still have running from the previous parts of this lab.

### Step 1: **Execute the bash script.**

Execute the bash script from the command line. You should see output similar to the following. After creating the **tempdir** directories, the script executes the commands to build the Docker container. Notice that Step 7/7 in the output executes the **sample_app.py** that creates the web server. Also, notice the container ID. You will see this in the Docker command prompt later in the lab.

```
devasc@labvm:~/labs/devnet-src/sample-app$ bash ./sample-app.sh
Sending build context to Docker daemon  6.144kB
Step 1/7 : FROM python
latest: Pulling from library/python
90fe46dd8199: Pulling fs layer
35a4f1977689: Pulling fs layer
bbc37f14aded: Pull complete
74e27dc593d4: Pull complete
4352dcff7819: Pull complete
deb569b08de6: Pull complete
98fd06fa8c53: Pull complete
```

```
7b9cc4fdefe6: Pull complete
512732f32795: Pull complete
Digest: sha256:ad7fb5bb4770e08bf10a895ef64a300b288696a1557a6d02c8b6fba98984b86a
Status: Downloaded newer image for python:latest
 ---> 4f7cd4269fa9
Step 2/7 : RUN pip install flask
 ---> Running in 32d28026afea
Collecting flask
  Downloading Flask-1.1.2-py2.py3-none-any.whl (94 kB)
Collecting click>=5.1
  Downloading click-7.1.2-py2.py3-none-any.whl (82 kB)
Collecting Jinja2>=2.10.1
  Downloading Jinja2-2.11.2-py2.py3-none-any.whl (125 kB)
Collecting Werkzeug>=0.15
  Downloading Werkzeug-1.0.1-py2.py3-none-any.whl (298 kB)
Collecting itsdangerous>=0.24
  Downloading itsdangerous-1.1.0-py2.py3-none-any.whl (16 kB)
Collecting MarkupSafe>=0.23
  Downloading MarkupSafe-1.1.1-cp38-cp38-manylinux1_x86_64.whl (32 kB)
Installing collected packages: click, MarkupSafe, Jinja2, Werkzeug,
itsdangerous, flask
Successfully installed Jinja2-2.11.2 MarkupSafe-1.1.1 Werkzeug-1.0.1
click-7.1.2 flask-1.1.2 itsdangerous-1.1.0
Removing intermediate container 32d28026afea
 ---> 619aee23fd2a
Step 3/7 : COPY  ./static /home/myapp/static/
 ---> 15fac1237eec
Step 4/7 : COPY  ./templates /home/myapp/templates/
 ---> dc807b5cf615
Step 5/7 : COPY  sample_app.py /home/myapp/
 ---> d4035a63ae14
Step 6/7 : EXPOSE 8080
 ---> Running in 40c2d35aa29a
Removing intermediate container 40c2d35aa29a
 ---> eb789099a678
Step 7/7 : CMD python3 /home/myapp/sample_app.py
 ---> Running in 41982e2c6209
Removing intermediate container 41982e2c6209
 ---> a2588e9b0593
Successfully built a2588e9b0593
Successfully tagged sampleapp:latest
8953a95374ff8ebc203059897774465312acc8f0ed6abd98c4c2b04448a56ba5
CONTAINER ID        IMAGE               COMMAND                  CREATED
STATUS                  PORTS                    NAMES
8953a95374ff        sampleapp           "/bin/sh -c 'python …"   1 second ago
Up Less than a second   0.0.0.0:8080->8080/tcp   samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ sudo bash ./sample-app.sh
Sending build context to Docker daemon  6.144kB
Step 1/7 : FROM python
latest: Pulling from library/python
4846f2900730: Pull complete
62fe2e747f69: Pull complete
4dc3950125eb: Pull complete
b2eb680928d4: Pull complete
9387c27cd4c4: Pull complete
28884091c75: Pull complete
34f44a36249d: Pull complete
Digest: sha256:16e7e5fc4e79fe153c818ac9b46595c7addddaad744d49595377836e300ad1c6
Status: Downloaded newer image for python:latest
 ---> c1046c84e051
Step 2/7 : RUN pip install flask
 ---> Running in 1efebbcb7bb2
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.4-cp315-cp315-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.9-py3-none-any.whl.metadata (4.1 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.4-cp315-cp315-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (23 kB)
Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask

Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.4 werkzeug-3.1.9
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering
your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if
ou know what you are doing and want to suppress this warning.
Removing intermediate container 1efebbcb7bb2
 ---> 7da906c0f534
Step 3/7 : COPY ./static /home/myapp/static/
 ---> 2365926911b5
Step 4/7 : COPY ./templates /home/myapp/templates/
 ---> d79f8b691352
Step 5/7 : COPY sample_app.py /home/myapp/
 ---> bf3416829df9
Step 6/7 : EXPOSE 8080
 ---> Running in da0c896fb50f
Removing intermediate container da0c896fb50f
```

```
Step 6/7 : EXPOSE 8080
 ---> Running in da0c896fb50f
Removing intermediate container da0c896fb50f
 ---> 44f509accee9
Step 7/7 : CMD python3 /home/myapp/sample_app.py
 ---> Running in 4af6da69bfe0
Removing intermediate container 4af6da69bfe0
 ---> 7f315edf287d
Successfully built 7f315edf287d
Successfully tagged sampleapp:latest
de54999e88fd8dba1bea83ef0238c22d10abb846972b90f05a7a2b105307e8ee
CONTAINER ID   IMAGE       COMMAND                  CREATED              STATUS              PORTS                                       NAMES
de54999e88fd   sampleapp   "/bin/sh -c 'python3…"   Less than a second ago   Up Less than a second   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   sample
running
```

## Step 2: **Investigate the running Docker container and the web app.**

a. The creation of the **tempdir** directories is not shown in the output for the script. You could add **echo** commands to print out messages when they are successfully created. You can also verify they are there with the **ls** command. Remember, this directory has the files and folders used to build the container and launch the web app. It is not the container that was built.

```
devasc@labvm:~/labs/devnet-src/sample-app$ ls tempdir/
Dockerfile  sample_app.py  static  templates
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ ls tempdir/
Dockerfile  sample_app.py  static  templates
```

b. Notice the Dockerfile created by your bash script. Open this file to see how it looks in its final form without the **echo** commands.

```
devasc@labvm:~/labs/devnet-src/sample-app$ cat tempdir/Dockerfile
FROM python
RUN pip install flask
COPY  ./static /home/myapp/static/
COPY  ./templates /home/myapp/templates/
COPY  sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
```

```
[analyst@secOps sample-app]$ cat tempdir/Dockerfile
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
```

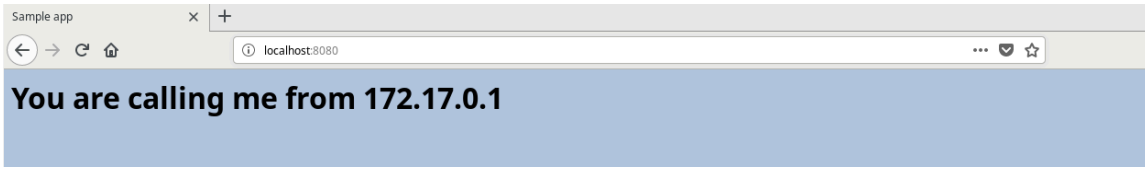
c. The output for the **docker ps -a** command may be hard to read depending on the width of your terminal display. You can redirect it to a text file where you can view it better without word wrapping.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a >> running.txt
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ docker ps -a >> running.txt
[analyst@secOps sample-app]$
```

d. The Docker container creates its own IP address from a private network address space. Verify the web app is running and reporting the IP address. In a web browser at **http://localhost:8080**, you should see the message **You are calling me from 172.17.0.1** formatted as H1 on a light steel blue background. You can also use the **curl** command, if you like.

```
devasc@labvm:~/labs/devnet-src/sample-app$ curl http://172.17.0.1:8080
<html>
<head>
    <title>Sample app</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>You are calling me from 172.17.0.1</h1>
</body>
</html>devasc@labvm:~/labs/devnet-src/sample-app$
devasc@labvm:~/labs/devnet-src/sample-app$
```



```
[analyst@secOps sample-app]$ curl http://172.17.0.1:8080
<html>
<head>
  <title>Sample app</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <h1>You are calling me from 172.17.0.1</h1>
</body>
</html>
```

e. By default, Docker uses the IPv4 172.17.0.0/16 subnet for container networking. (This address can be changed if necessary.) Enter the command **ip address** to display all the IP addresses used by your instance of the DEVASC VM. You should see the loopback address 127.0.0.1 that the web app used earlier in the lab and the new Docker interface with the IP address 172.17.0.1.

```
devasc@labvm:~/labs/devnet-src/sample-app$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
<output omitted>
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
group default
    link/ether 02:42:c2:d1:8a:2d brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:c2ff:fed1:8a2d/64 scope link
       valid_lft forever preferred_lft forever
<output omitted>
```

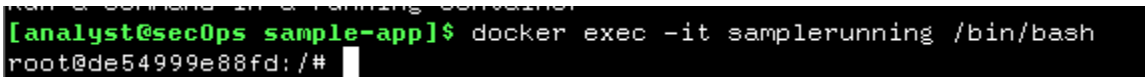
```
[analyst@secOps sample-app]$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fe:cb:5f brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.115/24 brd 192.168.100.255 scope global dynamic enp0s3
       valid_lft 77925sec preferred_lft 77925sec
    inet6 fe80::a00:27ff:fefe:cb5f/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:15:ed:d6:b2 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:15ff:feed:d6b2/64 scope link
       valid_lft forever preferred_lft forever
7: vethd9f13cd@if6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether fe:84:8e:1e:a5:d1 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::fc84:8eff:fe1e:a5d1/64 scope link
       valid_lft forever preferred_lft forever
```

## Step 3: Access and explore the running container.

Remember that a Docker container is a way of encapsulating everything you need to run your application so that it can easily be deployed in a variety of environments--not just in your DEVASC VM.

a. To access the running container, enter the **docker exec -it** command specifying the name of the running container (samplerunning) and that you want a bash shell (/bin/bash). The **-i** option specifies that you want it to be interactive and the **-t** option specifies that you want terminal access. The prompt changes to **root@containerID**. Your container ID will be different than the one shown below. Notice the container ID matches the ID shown in the output from **docker ps -a**.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker exec -it samplerunning /bin/bash
root@8953a95374ff:/#
```



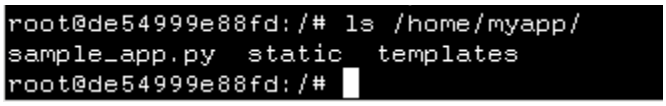

b. You are now in root access for the **samplerunning** Docker container. From here, you can use Linux commands to explore the Docker container. Enter **ls** to see the directory structure at the root level.

```
root@8953a95374ff:/# ls
bin   dev  home  lib64  mnt  proc  run   srv  tmp  var
boot  etc  lib   media  opt  root  sbin  sys  usr
root@8953a95374ff:/#
```




c. Recall that in your bash script, you added commands in the Dockerfile that copied your app directories and files to the **home/myapp** directory. Enter the **ls** command again for that folder to see your **sample_app.py** script and directories. To get a better understanding of what is included in your Docker container, you may wish to use the **ls** command to examine other directories such as /etc and /bin.

```
root@8953a95374ff:/# ls home/myapp/
sample_app.py  static  templates
root@8953a95374ff:/#
```




d. Exit the Docker container to return to the DEVASC VM command line.

```
root@8953a95374ff:/# exit
exit
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
root@de54999e88fd:/# exit
exit
[analyst@secOps sample-app]$
```

## Step 4: **Stop and remove the Docker container.**

a. You can stop the Docker container with the **docker stop** command specifying the name of the running container. It will take a few seconds to clean up and cache the container. You can see that it still exists by entering the **docker ps -a** command. However, if you refresh the web page for **http://localhost:8080**, you will see the web app is no longer running.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker stop samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a
CONTAINER ID        IMAGE               COMMAND                  CREATED
STATUS                      PORTS               NAMES
df034cb53e72        sampleapp           "/bin/sh -c 'python …"   49 minutes ago
Exited (137) 20 seconds ago                     samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ docker stop samplerunning
samplerunning
[analyst@secOps sample-app]$ docker ps -a
CONTAINER ID   IMAGE       COMMAND                  CREATED          STATUS
                 PORTS     NAMES
de54999e88fd   sampleapp   "/bin/sh -c 'python3…"   16 minutes ago   Exited (137
) 12 seconds ago             samplerunning
```

b. You can restart a stopped container with the **docker start** command. The container will immediately spin up.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker start samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ docker start samplerunning
samplerunning
```

c. To permanently remove the container, first stop it and then remove it with the **docker rm** command. You can always rebuild it again executing the **sample-app** program. Use the **docker ps -a** command to verify the container has been removed.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker stop samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker rm samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS              PORTS               NAMES
devasc@labvm:~/labs/devnet-src/sample-app$
```

```
[analyst@secOps sample-app]$ docker stop samplerunning
samplerunning
[analyst@secOps sample-app]$ docker rm samplerunning
samplerunning
[analyst@secOps sample-app]$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[analyst@secOps sample-app]$
```

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**Conclusion**

In this activity, I learned how Docker and web applications can be used to deploy an application in a containerized environment. Through this activity, I learned how to build a basic web application using Python and Flask and connect it with HTML and CSS files. I also gained experienced in creating a Dockerfile and using bash commands to build and run a docker container. The activity helped me understand how docker packages an application and its required files so it can run in an isolated environment. Overall, I learned how bash scripting, flask and docker can work together to create and deploy a simple web application.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.